CUET PG

DAILY CLASS NOTES

Computer Science

Operating System

Lecture – 15

Page Replacement Algorithm



Page Replacement Algorithm

Topics to be Covered

1. **Page Replacement Algorithm**
   (a) First-In-First-Out (FIFO)
   (b) Optimal Replacement
   (c) Least Recently Used (LRU)
2. **Locality of Reference**
3. **Thrashing**

1. Page Replacement Algorithms

When a required page is not present in main memory, a **page fault** occurs.

If no free frame is available, the system must **replace an existing page** using one of the following algorithms.

(a) First-In-First-Out (FIFO) Algorithm

- The **oldest page** in memory (the one loaded first) is selected for replacement.
- Implemented using a **queue** structure.
- Simple, but may cause **Belady's Anomaly** — increasing frames can **increase page faults**.

(b) Optimal Page Replacement

- This algorithm gives the **lowest possible page fault rate** among all algorithms.
- The page chosen for replacement is the one **that will not be used for the longest time in the future**.
- Requires **future knowledge** of reference string (impractical for real-time systems but useful as a theoretical benchmark).
- Used to measure the efficiency of other algorithms.

(c) Least Recently Used (LRU) Algorithm

- The page **not used for the longest period of time** in the past is selected for replacement.
- Based on the **principle of temporal locality** (recently used pages are likely to be used again).
- LRU can be implemented using:
  - **Counters** (record last used time)
  - **Stacks** (to keep the order of page usage)
- Produces good results in practice, though more complex than FIFO.

2. Locality of Reference

Definition

- During execution, a process accesses only a **small subset** of its total logical address space at a time.
- This subset is called its **current locality of reference**.
- As execution proceeds, the **current locality keeps shifting** (e.g., from code to data to stack areas).

Significance

- This concept supports **paging and demand paging**, because only the current locality needs to be in main memory at any moment.
- Reduces total memory requirements and page faults.

3. Thrashing

Definition

- **Thrashing** occurs when the system spends **most of its time swapping pages in and out of memory**, rather than executing processes.
- This excessive paging causes **poor CPU utilization** and **slow system performance**.

Explanation

- When the degree of multiprogramming increases:
  - Initially, CPU utilization increases (more processes are ready).
  - After a certain limit, if too many processes are competing for limited frames → **page faults rise sharply**, leading to **thrashing**.
  - At this point, **CPU utilization drops drastically**.
- This relation is often shown in a graph of CPU Utilization vs. Degree of Multiprogramming (Photo 1 on page 11).

4. Measures to Avoid Thrashing

Page-Fault Frequency (PFF) Strategy

- A dynamic approach to control thrashing by **monitoring page-fault rate**.
- **Steps:**
  1. Define **upper and lower bounds** for the acceptable page-fault rate.
  2. If a process's page-fault rate **exceeds the upper limit**, allocate **more frames** to it.
  3. If the rate **falls below the lower limit**, **remove some frames** from it.
  4. Thus, the page-fault rate is directly controlled to **prevent thrashing**.

Summary

Concept	Description
Page Replacement Algorithms	Used when no free frame is available to replace pages in main memory.
FIFO	Replaces the oldest page in memory.
Optimal	Replaces the page that will not be used for the longest time in the future.
LRU	Replaces the page least recently used.
Locality of Reference	Process works on a small subset of memory at a time; locality changes over time.
Thrashing	Occurs when the system spends more time in paging than processing.
PFF (Page-Fault Frequency)	Maintains page-fault rate within limits by dynamically adjusting frames per process.

पेज रिप्लेसमेंट एल्गोरिथम

कवर किए जाने वाले विषय

1. **पेज रिप्लेसमेंट एल्गोरिदम**
   (a) फर्स्ट-इन-फर्स्ट-आउट (FIFO)
   (b) ऑप्टिमल रिप्लेसमेंट
   (c) लीस्ट रिसेंटली यूज्ड (LRU)
2. **रेफरेंस की लोकैलिटी**
3. **थ्रैशिंग**

1. पेज रिप्लेसमेंट एल्गोरिथम

जब मुख्य मेमोरी में आवश्यक पेज मौजूद नहीं होता है, तो पेज फॉल्ट होता है।

यदि कोई फ्री फ्रेम उपलब्ध नहीं है, तो सिस्टम को निम्नलिखित एल्गोरिदम में से किसी एक का उपयोग करके मौजूदा पेज को बदलना होगा।

(a) फर्स्ट-इन-फर्स्ट-आउट (FIFO) एल्गोरिथम

- मेमोरी में मौजूद सबसे पुराने पेज (जो सबसे पहले लोड हुआ था) को प्रतिस्थापन के लिए चुना जाता है।
- इसे क्यू स्ट्रक्चर का उपयोग करके कार्यान्वित किया जाता है।
- यह सरल है, लेकिन इससे बेलाडी की विसंगति उत्पन्न हो सकती है — फ्रेम की संख्या बढ़ने से पेज फॉल्ट की संख्या भी बढ़ सकती है।

(b) ऑप्टिमल पेज रिप्लेसमेंट

- यह एल्गोरिथम सभी एल्गोरिथमों में सबसे कम पेज फॉल्ट दर प्रदान करता है।
- प्रतिस्थापन के लिए चुना गया पेज वह है जिसका उपयोग भविष्य में सबसे लंबे समय तक नहीं किया जाएगा।
- इसके लिए संदर्भ स्ट्रिंग की भविष्य की जानकारी आवश्यक है (वास्तविक समय प्रणालियों के लिए अव्यावहारिक, लेकिन सैद्धांतिक मापदंड के रूप में उपयोगी)।
- इसका उपयोग अन्य एल्गोरिथमों की दक्षता मापने के लिए किया जाता है।

(c) लिस्ट रिसेंटली यूज्ड (LRU) एल्गोरिथम

- अतीत में सबसे लंबे समय तक अप्रयुक्त रहे पेज को प्रतिस्थापन के लिए चुना जाता है।
- यह अस्थायी स्थानीयता के सिद्धांत पर आधारित है (हाल ही में उपयोग किए गए पेज के दोबारा उपयोग किए जाने की संभावना अधिक होती है)।
- एलआरयू को निम्न प्रकार से कार्यान्वित किया जा सकता है:
  - काउंटर (अंतिम उपयोग समय रिकॉर्ड करने के लिए)
  - स्टैक (पेज के उपयोग का क्रम बनाए रखने के लिए)
- प्रोड्यूस में यह अच्छे परिणाम देता है, हालांकि यह FIFO की तुलना में अधिक जटिल है।

2. संदर्भ का स्थान

परिभाषा

- एग्जीक्यूशन के दौरान, एक प्रक्रिया एक समय में अपने कुल लॉजिकल एड्रेस के केवल एक छोटे से उपसमूह तक ही पहुँच पाती है।
- इस सबसेट को इसकी वर्तमान संदर्भ स्थिति कहा जाता है।
- एग्जीक्यूशन आगे बढ़ने के साथ-साथ, वर्तमान संदर्भ स्थिति बदलती रहती है (उदाहरण के लिए, कोड से डेटा और स्टैक क्षेत्रों तक)।

महत्व

- यह अवधारणा पेजिंग और डिमांड पेजिंग को सपोर्ट करती है, क्योंकि किसी भी समय केवल वर्तमान लोकैलिटी को ही मुख्य मेमोरी में रहने की आवश्यकता होती है।
- इससे कुल मेमोरी की आवश्यकता और पेज फॉल्ट कम हो जाते हैं।

3. थ्रेशिंग

परिभाषा

- थ्रैशिंग तब होती है जब सिस्टम अपना अधिकांश समय प्रक्रियाओं को एग्जीक्यूशन करने के बजाय मेमोरी में पेजों को स्वैप करने में व्यतीत करता है।
- इस अत्यधिक पेजिंग के कारण सीपीयू का उपयोग कम होता है और सिस्टम का प्रदर्शन धीमा हो जाता है।

स्पष्टीकरण

- जब मल्टीप्रोग्रामिंग की मात्रा बढ़ती है:
  - शुरुआत में, सीपीयू का उपयोग बढ़ता है (अधिक प्रक्रियाएं तैयार होती हैं)।
  - एक निश्चित सीमा के बाद, यदि बहुत सारी प्रक्रियाएं सीमित फ्रेम के लिए प्रतिस्पर्धा करती हैं → पेज फॉल्ट तेजी से बढ़ते हैं,
  - जिससे थ्रैशिंग होती है।
  - इस बिंदु पर, सीपीयू का उपयोग तेजी से घट जाता है।
- इस संबंध को अक्सर सीपीयू उपयोग बनाम मल्टीप्रोग्रामिंग की मात्रा के ग्राफ में दर्शाया जाता है (पृष्ठ 11 पर फोटो 1)।

4. थ्रैशिंग से बचने के उपाय

पेज-फॉल्ट आवृत्ति (PFF) रणनीति

- पेज-फॉल्ट दर की निगरानी करके थ्रैशिंग को नियंत्रित करने का एक गतिशील दृष्टिकोण।
- **चरण:**
  1. स्वीकार्य पेज-फॉल्ट दर के लिए ऊपरी और निचली सीमाएँ निर्धारित करें।
  2. यदि किसी प्रक्रिया की पेज-फॉल्ट दर ऊपरी सीमा से अधिक हो जाती है, तो उसे अधिक फ्रेम आवंटित करें।
  3. यदि दर निचली सीमा से कम हो जाती है, तो उससे कुछ फ्रेम हटा दें।
  4. इस प्रकार, थ्रैशिंग को रोकने के लिए पेज-फॉल्ट दर को सीधे नियंत्रित किया जाता है।

सारांश

अवधारणा	विवरण
पेज रिप्लेसमेंट एल्गोरिथम	इसका उपयोग तब किया जाता है जब मुख्य मेमोरी में पेजों को बदलने के लिए कोई फ्री फ्रेम उपलब्ध नहीं होता है।
FIFO	मेमोरी में मौजूद सबसे पुराने पेज को बदल देता है।
ऑप्टीमल	यह उस पेज को बदल देता है जिसका उपयोग भविष्य में सबसे लंबे समय तक नहीं किया जाएगा।
LRU	यह सबसे कम उपयोग किए गए पेज को बदल देता है।
संदर्भ का स्थान	यह प्रक्रिया एक समय में मेमोरी के एक छोटे से हिस्से पर काम करती है; मेमोरी का स्थान समय के साथ बदलता रहता है।
थ्रैशिंग	यह तब होता है जब सिस्टम प्रोसेसिंग की तुलना में पेजिंग में अधिक समय व्यतीत करता है।
PFF (पेज-फॉल्ट फ्रीक्वेंसी)	प्रत्येक प्रक्रिया के लिए फ्रेम को गतिशील रूप से समायोजित करके पेज-फॉल्ट दर को सीमा के भीतर बनाए रखता है।



PW Web/App: https://smart.link/7wwosivoicgd4
Library- https://smart.link/sdfez8ejd80if